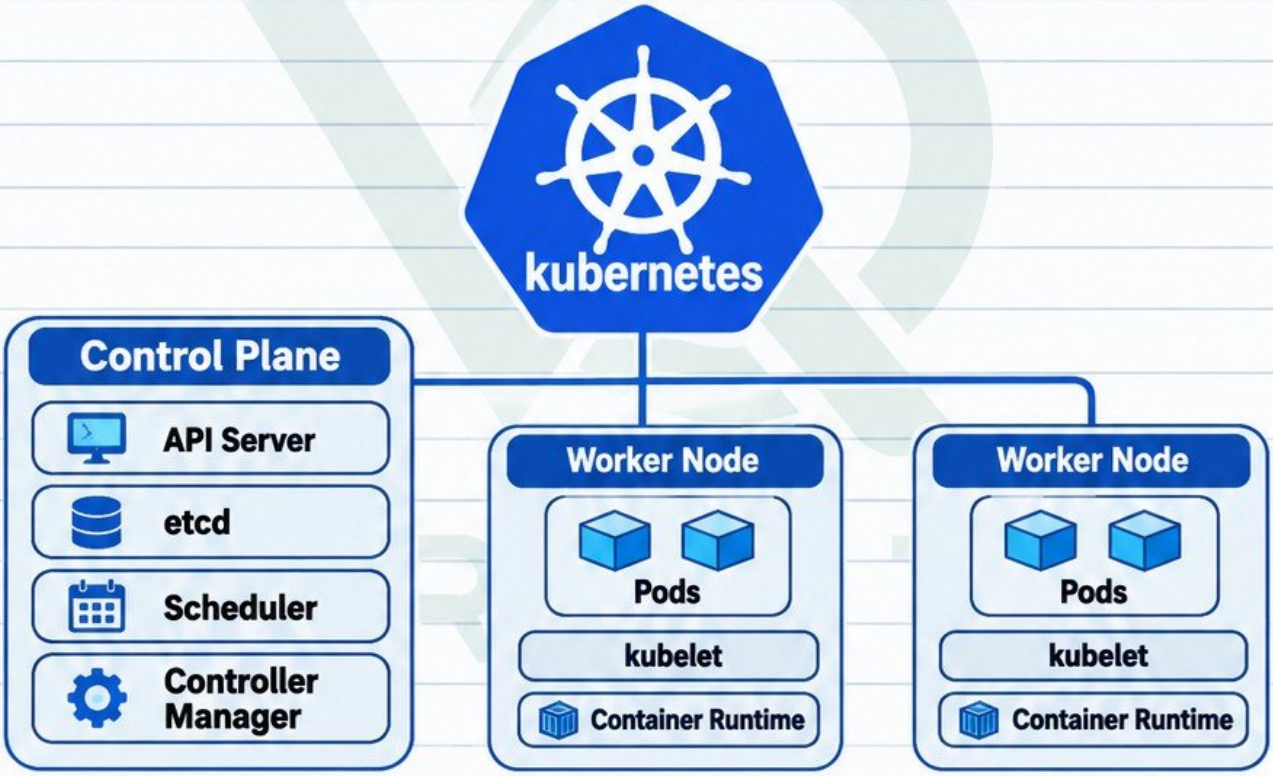
VERIQTA

THE KUBERNETES HANDBOOK

A Practical Guide for Junior DevOps, Cloud and Platform Engineers



What You Will Learn

- ✓ How Kubernetes Works
- ✓ Deploy and Scale Applications
- ✓ Networking and Ingress
- ✓ Configuration and Storage
- ✓ Security and Access Control
- ✓ Troubleshooting Real Failures



@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

Kubernetes: The Problem It Solves

What Is Kubernetes?

Kubernetes (K8s) is a container orchestration platform that automatically runs, manages, scales, and keeps your containerized applications available and reliable.

The Problem With Manual Containers

- Works for small apps, but becomes hard as the app grows.
- You must start, stop, and restart containers yourself.
- No automatic recovery if a container crashes.
- Hard to scale when traffic increases.
- Difficult to keep multiple servers consistent.

How Kubernetes Helps

- **Scheduling** – places containers on the right servers (nodes).
- **Self-healing** – automatically restarts failed containers.
- **Scaling** – increases or decreases the number of containers.
- **High availability** – keeps your app running with minimal downtime.

Manual Container vs. Kubernetes-Managed Workload

Manual Management



- You manage start/stop/restart.
- No automatic recovery.
- Scaling is manual and slow.
- Higher risk of downtime.

With Kubernetes



- Automatically schedules containers.
- Restarts failed containers (self-healing).
- Scales up or down based on demand.
- Keeps the application highly available.

Junior Engineer Tip

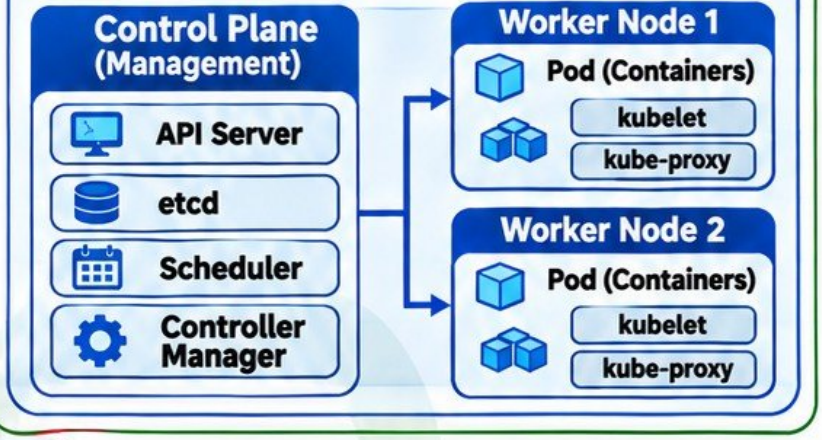
Use **Kubernetes** when you need to run containerized applications reliably, at scale, and with minimal manual work.

Kubernetes Architecture: Inside a Cluster

Overview

- Kubernetes (K8s) runs a cluster with two main parts:
  - **Control Plane** – the brain that manages the cluster.
  - **Worker Nodes** – the machines that run your application containers (Pods).

Cluster Architecture



Key Components and Roles

API Server	The central entry point (API) that receives requests (e.g., create a Deployment).
etcd	A key-value database that stores the cluster state (everything about your cluster).
Scheduler	Decides which worker node should run a new Pod.
Controller Manager	Runs controllers (e.g., keeps the desired number of Pods running, restarts failed Pods).
kubelet	An agent on each worker node that talks to the API server and runs containers.
Container Runtime	Runs the containers (e.g., containerd, CRI-O, Docker).
kube-proxy	Handles networking and service access to reach your Pods (load balancing).

How a Workload Becomes a Running Container

1. You submit a workload (e.g., Deployment) using **kubectl**.
2. API Server validates the request and saves the desired state in **etcd**.
3. Scheduler selects a suitable worker node.
4. Controller Manager ensures the required Pods are created and kept running.
5. **kubelet** on the chosen node uses the **container runtime** to start the container, and **kube-proxy** makes it reachable.

Junior Engineer Tip

Use **kubectl get**, **describe**, and **logs** to see what the control plane decided and what is running on the worker node.

Your First Kubernetes Cluster

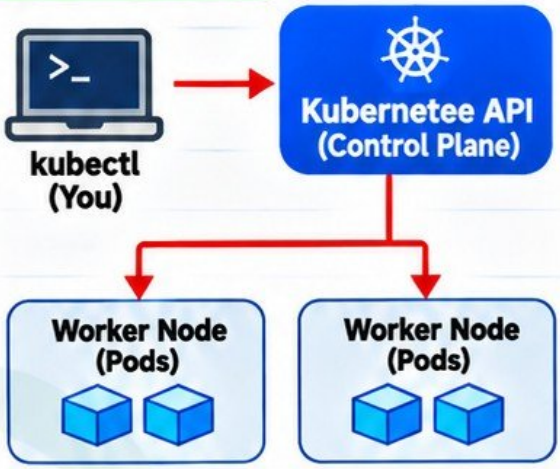
Key Tools and Concepts

kubectl | The command-line tool used to talk to the Kubernetes API.

kind | A tool that runs a Kubernetes cluster on your local machine (using Docker).

Kubernetes API | The entry point for managing the cluster (e.g., create, get, delete resources).

How It Works



Hands-On: Create and Explore a Local Cluster

1. Create a cluster with kind.

```
kind create cluster --name veriqta-k8s
```

Output: Creating cluster "veriqta-k8s" ...
✓ Done

2. Check the cluster nodes.

```
kubectl get nodes
```

Output:

NAME	STATUS	ROLES	AGE
veriqta-k8s-control-plane	Ready	control-plane	1m

3. View namespaces (defaults).

```
kubectl get namespaces
```

Output:

NAME	STATUS	AGE
default	Active	1m
kube-system	Active	1m
kube-public	Active	1m

4. Inspect cluster information.

```
kubectl cluster-info
```

Output:
Kubernetes control plane is running at https://127.0.0.1:xxxex
CoreDNS is running at https://127.0.0.1:xxxex/api/v1/namespaces/kube-system/services/kube-dns:dns

What Is a kubeconfig File?



A configuration file (usually at ~/.kube/config) that stores your cluster connection details, credentials, and contexts.

Why Check Your Cluster Context?

- You can have multiple clusters (e.g., local, dev, staging, prod).
- Running a command in the wrong cluster can change or delete production resources.
- Always verify where you are before making changes.

Check and Switch Context

```
kubectl config current-context
```

Output: veriqta-k8s

```
kubectl config get-contexts
```

```
kubectl config use-context veriqta-k8s
```

Junior Engineer Tip

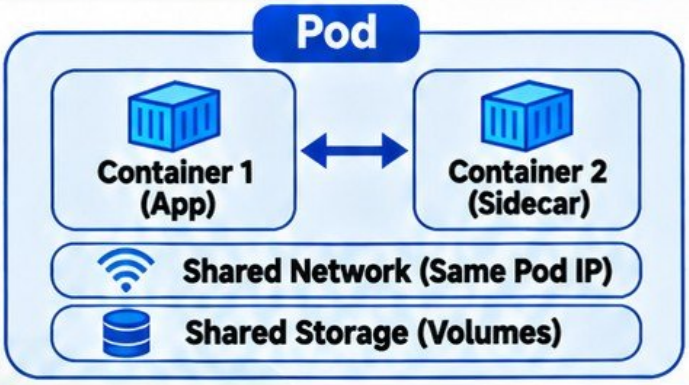
Always run **kubectl config current-context** before creating, updating, or deleting resources.

Pods: Where Applications Actually Run

What Is a Pod?

- A Pod is the smallest deployable unit in Kuberentes.
- It runs one or more containers that work together.
- Containers in the same Pod share the same network (IP address) and storage (volumes).

Pod Structure



Create a Pod (YAML)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx
    image: nginx:1.27
    ports:
    - containerPort: 80
```

YAML

Hands-On: Deploy and Inspect the Pod

1	`kubectl apply -f nginx-pod.yaml`	Create the Pod from the YAML file.
2	`kubectl get pods`	Check the Pod status.
3	`kubectl describe pod nginx-pod`	View detailed info (events, IP, etc.).
4	`kubectl logs nginx-pod`	Read the application logs from the container.

Example Output

```
$ kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
nginx-pod   1/1     Running   0          1m
```

```
$ kubectl describe pod nginx-pod
Pod IP: 10.244.0.12
Events:
  Normal  Pulled   Successfully pulled image
  Normal  Started  Started container nginx
```

Why Not Use Standalone Pods in Production?

- Pods can be deleted or lost easily.
- They are not automatically restarted with your desired number of replicas.
- They do not provide scaling or updates.
- Use Deployments (next) to manage Pods reliably in production.

Junior Engineer Tip

Use Pods to run and test containers, but use **Deployments** to keep your application running, scalable, and resilient in production.

Deployments: Keeping Applications Running

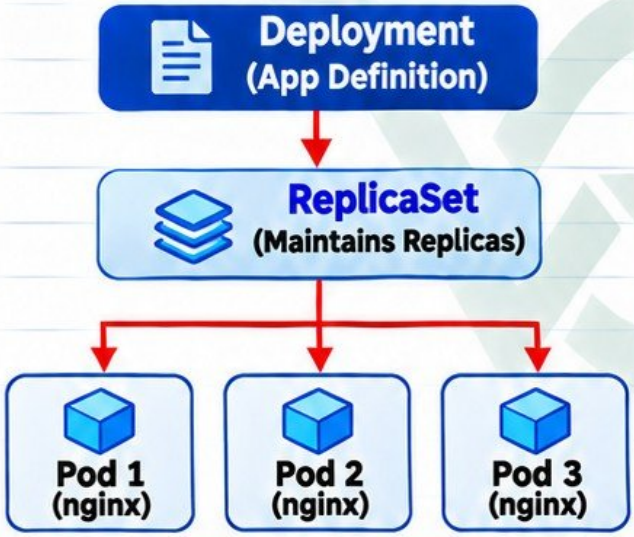
What Is a Deployment?

- A **Deployment** is a Kubernetes resource that manages your application and keeps the **desired state** (e.g., 3 replicas) running at all times.

Key Terms

Deployment	Manages the application and desired state (replicas, updates, rollbacks).
ReplicaSet	Ensures the correct number of identical Pods are running.
Desired State	The target state you want (e.g., 3 running Pods).

How They Work Together



Create a Deployment (NGINX)

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports:
        - containerPort: 80
```

Deploy and Verify

`$ kubectl apply -f nginx-deploy.yaml`	Create the Deployment.
`$ kubectl get deployments`	Check the Deployment.
`$ kubectl get rs`	View the ReplicaSet.
`$ kubectl get pods -l app=nginx`	See the 3 running Pods.

What Happens When a Pod Fails?

1. A Pod crashes or is deleted.
2. The ReplicaSet detects the missing Pod.
3. Kubernetes automatically creates a new Pod to keep 3 replicas running.

Junior Engineer Tip

Use **Deployments** (not standalone Pods) to keep your application available, self-healing, and easy to scale and update.

Services: How Pods Communicate

Why Do We Need a Service?

- Pod IP addresses are **temporary** and can change when a Pod restarts, is rescheduled, or scaled.
- A Service gives a **stable address** to reach a group of healthy Pods, even when their IPs change.

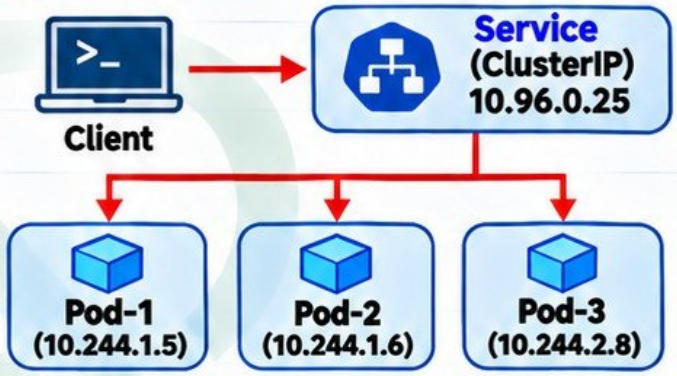
Types of Services

ClusterIP	Default type. Private access inside the cluster (Pod-to-Pod).
NodePort	Exposes the app on a port on each node (e.g., <NodeIP>:30080).
LoadBalancer	Provision a cloud load balancer for external access (e.g., AWS, GCP).

Key Concepts

Labels	Key-value tags on resources (e.g., app=nginx).
Selectors	Rules a Service uses to find matching Pods (e.g., app=nginx).
Endpoints	The list of healthy Pod IPs that the Service forwards traffic to.

How Traffic Flows to Pods (ClusterIP)



Hands-On: Expose a Deployment with a Service (ClusterIP)

1 Deploy an NGINX application.

```
kubectl create deployment nginx \
 --image=nginx:1.27  --replicas=3
```

2 Create a ClusterIP Service.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
   - port: 80
     targetPort: 80
```

YAML

Verify and Explore

```
$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	PORT(S)
nginx-svc	ClusterIP	10.96.0.25	80/TCP

```
$ kubectl get endpoints nginx-svc
```

NAME	ENDPOINTS
nginx-svc	10.244.1.5:80, 10.244.1.6:80, 10.244.2.8:80

```
$ kubectl get pods -l app=nginx
```

NAME	READY	STATUS	IP
nginx-7d9f6c-f8b2d	1/1	Running	10.244.1.5
nginx-7d9f6c-gt9k2	1/1	Running	10.244.1.6
nginx-7d9f6c-mq4jn	1/1	Running	10.244.2.8

Junior Engineer Tip

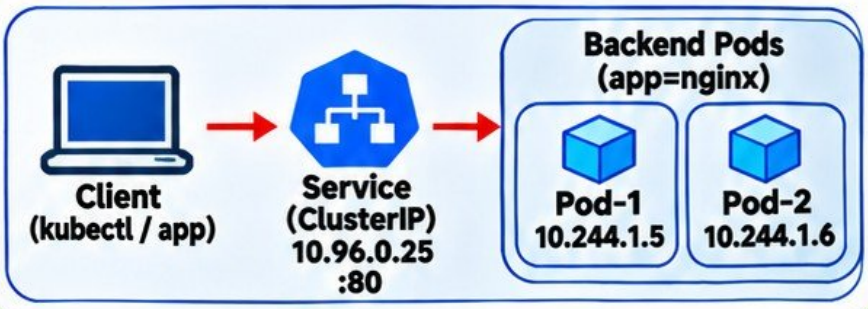
Use **Services** to give your application a stable, reliable endpoint, and let Kubernetes automatically route traffic only to healthy Pods.

Kubernetes Networking: Follow One Request

Overview

- Let's follow a real request from a client to an application Pod.
- We will see how a Service routes traffic, how Pods communicate, and the role of DNS, ports, and Service discovery.

Request Flow: Client → Service → Pod



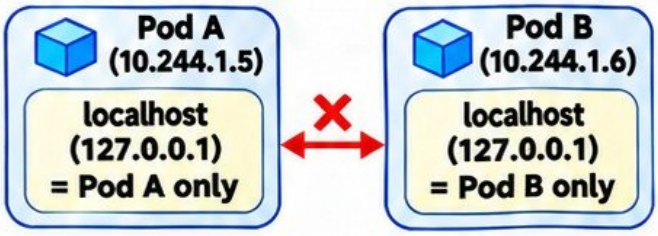
Key Networking Concepts

Cluster Networking	A flat, routable network that lets all Pods communicate.
Pod-to-Pod	Pods can talk to each other using their Pod IPs.
DNS	Built-in DNS (CoreDNS) lets you reach a Service by name (e.g., nginx-svc).
Ports	Define the port your app listens on (e.g., 80, 8080).
Service Discovery	Finds healthy Pods behind a Service using labels and selectors.

The Traffic Path (Step by Step)

1. Client sends a request to the Service (e.g., http://nginx-svc:80).
2. Cluster DNS resolves nginx-svc to the Service IP (e.g., 10.96.0.25).
3. The Service selects healthy Pods (e.g., app=nginx).
4. Traffic is load-balanced to one of the Pods (e.g., 10.244.1.5).
5. The Pod receives the request on the target port (e.g., 80) and returns a response.

Why localhost Does Not Reach Another Pod



- localhost (127.0.0.1) always points to the current Pod itself, not other Pods.
- To reach another Pod, use its Pod IP or a Service DNS name (e.g., nginx-svc).

Hands-On: Connectivity Test

`$ kubectl get svc`	Check the Service and its ClusterIP.
`$ kubectl get pods -l app=nginx`	List the application Pods and their IPs.
`$ kubectl exec -it pod-a -- curl http://nginx-svc:80`	Test access to the app via the Service (expected: NGINX welcome page).
`$ kubectl exec -it pod-a -- curl http://10.244.1.6:80`	Test direct Pod-to-Pod access using the Pod IP.
`$ kubectl exec -it pod-a -- curl http://localhost:80`	Only works if the app is running inside pod-a.

Junior Engineer Tip

Use the Service DNS name (e.g., **nginx-svc**) for stable communication between applications, not Pod IPs or localhost.

Ingress and Gateway: Exposing Web Applications

Overview

- Lets external users (HTTP/HTTPS) reach applications running inside the cluster.
- Uses an **Ingress** (or **Gateway**) combined with a **controller** to route traffic to the right Service and Pods.

Key Terms

Ingress	A Kubernetes resource that defines HTTP/HTTPS routing rules (e.g., host, path) to Services.
Ingress Controller	A controller (e.g., NGINX, Traefik) that watches Ingress resources and implements the routing (data plane).
Gateway API	A newer, more flexible API (Gateway, HTTPRoute) for advanced, production-ready traffic management.

Request Flow: From Internet to Pods



Host-Based Routing Example

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: veriqta-ingress
spec:
  ingressClassName: nginx
  rules:
  - host: web.veriqta.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-svc
            port:
              number: 80
  - host: api.veriqta.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: api-svc
            port:
              number: 80
```

YAML

How It Works

1. DNS resolves web.veriqta.local to the Ingress controller (public IP).
2. The controller reads the Ingress rules.
3. It routes the request to the matching Service (web-svc or api-svc).
4. The Service load-balances the traffic to healthy Pods.

Important Notes

- An Ingress resource alone does NOT create a working controller or public endpoint.
- You must install an Ingress controller (e.g., NGINX Ingress, Traefik, or cloud controller).
- A public endpoint usually comes from a LoadBalancer Service, cloud integration, or an external IP.
- Use the Gateway API when you need more advanced routing, multiple teams, or richer traffic policies.

Test the Setup

`$ kubectl get ingress`	Check the Ingress resource.
`$ kubectl get pods -n ingress-nginx`	Verify the controller is running.
Test in browser:	http://web.veriqta.local

Junior Engineer Tip

To expose applications, you need three things working together: an **Ingress** (or **Gateway**), a running **controller**, and a reachable **public endpoint** (e.g., LoadBalancer or DNS).

ConfigMaps and Secrets: Application Configuration

Overview

- Provide configuration and sensitive data to applications without rebuilding container images.
- Use ConfigMaps for non-sensitive settings and Secrets for sensitive data (e.g., passwords, API keys).

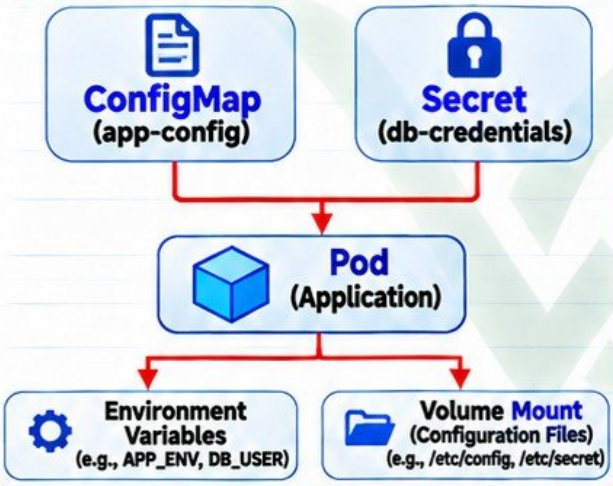
Key Benefits

- Change config without a new image.
- Reuse the same image across environments.
- Keep sensitive data out of code and images.

ConfigMap vs Secret

ConfigMap	Stores non-sensitive configuration (e.g., app settings).
Secret	Stores sensitive data (e.g., passwords, tokens, API keys).
Key Point	Secrets are not automatically encrypted in etcd; use proper access controls.

How It Works



Create a ConfigMap (Example)

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"
  API_URL: "https://api.veriqta.local"
```

YAML

Create a Secret (Example)

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  DB_USER: YWRtaW4=
  DB_PASSWORD: U3VpZXJQYXNzMTIz
```

YAML

Note: Values are base64-encoded, not fully encrypted.

Use in a Pod: Env Vars and Volume Mounts

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-with-config
spec:
  containers:
    - name: app
      image: nginx:1.27
      eny:
        - name: APP_ENV
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: APP_ENV
        - name: DB_USER
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: DB_USER
      volumeMounts:
        - name: config-vol
          mountPath: /etc/config
  volumes:
    - name: config-vol
      configMap:
        name: app-config
```

YAML

Hands-On: Create and Verify

`$ kubectl apply -f app-config.yaml`	Create the ConfigMap.
`$ kubectl apply -f db-secret.yaml`	Create the Secret.
`$ kubectl apply -f app-pod.yaml`	Deploy the Pod using both.
`$ kubectl get configmap,secret`	Verify they exist.
`$ kubectl exec -it app-with-config -- env | grep APP_ENV`	Check the environment variable from the ConfigMap.
`$ kubectl exec -it app-with-config -- ls /etc/config`	Verify the mounted configuration file.

Security Notes

- Kubernetes Secrets are not automatically encrypted in etcd.
- Use RBAC (least privilege) to restrict who can read them.
- Enable etcd encryption at rest (e.g., EncryptionConfiguration).
- Consider external secret managers (e.g., AWS Secrets Manager, Vault) for higher security in production.

Junior Engineer Tip

Use ConfigMaps for non-sensitive settings and Secrets for sensitive data, and always protect Secrets with RBAC and etcd encryption in production.

Persistent Storage: Keeping Data After Pod Restarts

Overview

- Containers have **ephemeral storage**, which is lost when the Pod is deleted or recreated.
- Use **PersistentVolumes (PV)**, **PersistentVolumeClaims (PVC)**, and **StorageClasses** to keep data safe and available.

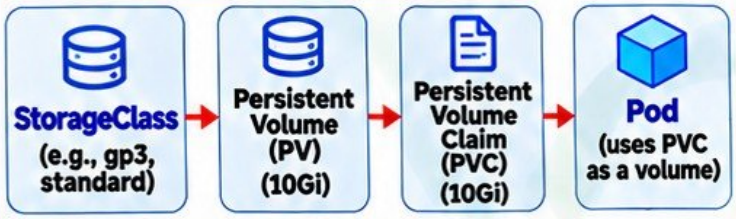
Key Terms

Ephemeral Storage	Temporary storage inside a container; lost when the Pod is gone.
PersistentVolume (PV)	A piece of storage provisioned in the cluster (e.g., a disk).
PersistentVolume Claim (PVC)	A request for storage by an application (e.g., "give me 10Gi").
StorageClass	Defines how storage is provisioned (e.g., AWS EBS, GCE PD, local).

Why It Matters

- Keeps application data (e.g., databases, uploads) across Pod restarts, rescheduling, or updates.
- Separates application lifecycle (Pods) from data lifecycle (storage).

Storage Flow (How It Works)



Ephemeral vs Persistent Storage

Ephemeral (Container Storage)
- Fast and simple.
- Lost when the Pod is deleted.
- Not for important data.

Persistent (PV + PVC)
- Survives Pod restarts.
- Keeps your application data (e.g., database, files).
- Backed by cluster storage (local, cloud, or network).

Create Storage and Attach to a Workload

1. StorageClass (dynamic storage)

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: app-storage
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp3
reclaimPolicy: Retain
```

2. PersistentVolumeClaim

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-pvc
spec:
  storageClassName: app-storage
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 10Gi
```

3. Deployment (mount the PVC)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      containers:
      - name: app
        image: nginx:1.27
        volumeMounts:
        - name: data
          mountPath: /usr/share/nginx/html
      volumes:
        - name: data
          persisstentVolumeClaim:
            claimName: app-pvc
```

Hands-On: Apply and Verify

`$ kubectl apply -f storageclass.yaml`	Create the StorageClass.
`$ kubectl apply -f pvc.yaml`	Create the PVC.
`$ kubectl apply -f deployment.yaml`	Deploy the app with the volume.
`$ kubectl get pvc`	Verify the PVC is bound.
`$ kubectl get pv`	Check the PersistentVolume.
`$ kubectl exec -it deploy/app -- sh -c 'echo "Hello VERIQTA" > /usr/share/nginx/html/index.html"`	Write a test file to the volume.
`$ kubectl exec -it deploy/app -- cat /usr/share/nginx/html/index.html`	Confirm the data exists.

Keep Data After a Pod Restart



- The Pod can be deleted or recreated (e.g., during an update).
- The data remains because it is stored on a PersistentVolume via a PVC.

Important Notes and Limitations

- Deleting a Pod does NOT delete the PersistentVolume or PVC.
- Data is retained based on the PV's reclaim policy:
  - **Retain** – keep the data (manual cleanup).
  - **Delete** – automatically remove the storage (common in dynamic storage).
  - **Recycle** – deprecated (not recommended).
- Persistent storage is not a backup. Use regular backups, snapshots, or a data protection strategy for disaster recovery.

Junior Engineer Tip

Store important application data on a **PersistentVolume** (via PVC) so it survives Pod restarts, updates, and rescheduling.

CPU and Memory: Requests, Limits, and Scheduling

Overview

- Control how much **CPU** and **memory** a container can use.
- **Requests** tell the scheduler the minimum resources needed.
- **Limits** set the maximum resources a container can use.

Key Terms

Request	The minimum CPU/memory reserved for scheduling (e.g., 250m, 512Mi).
Limit	The maximum CPU/memory a container can use (e.g., 1, 1Gi).
CPU (vCPU)	Compute power (e.g., 250m = 0.25 core, 1 = 1 core).
Memory	RAM (e.g., 512Mi, 1Gi).

Why It Matters

- Helps the scheduler place Pods on nodes with enough capacity.
- Prevents one app from consuming all resources.
- Improves stability, performance, and reliability.

How the Scheduler Uses Requests



- The scheduler checks node capacity and places the Pod on a node that can satisfy the requests.
- Requests reserve resources on the node for the Pod.

CPU Throttling vs Memory OOMKilled

CPU Throttling

- Happens when the container tries to use more CPU than its limit.
- Kubernetes (cgroups) slows it down (throttles) to keep usage within the limit.
- The Pod usually keeps running, but performance drops (e.g., higher response time).

OOMKilled (Memory)

- Happens when the container uses more memory than its limit.
- The kernel kills the container to prevent node memory exhaustion.
- The Pod restarts and shows the reason **OOMKilled** in events and status.

Example: Resource Configuration (NGINX)

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-resources
spec:
  replicas: 2
  template:
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        resources:
          requests:
            cpu: "250m"      # 0.25 CPU
            memory: "512Mi"  # 512 MB
          limits:
            cpu: "1"         # 1 CPU
            memory: "1Gi"    # 1 GB
```

Inspect and Troubleshoot

`$ kubectl describe pod nginx-resources-xxx`	View requests, limits, and events (e.g., OOMKilled).
`$ kubectl top pod`	Check current CPU and memory usage (Metrics Server).
`$ kubectl top node`	See node resource usage and capacity.
`$ kubectl get pod -o wide`	Check Pod placement (node it's running on).
`$ kubectl get events --sort-by=.lastTimestamp`	Find scheduling, throttling, or OOMKilled events.

Real-World Example



- **Request:** CPU: 250m, Memory: 512Mi (guarantees scheduling).
- **Limit:** CPU: 1, Memory: 1Gi (maximum the container can use).
- **Result:** Runs reliably without consuming excessive node resources.

Common Failure Signals

Symptom	Likely Cause	What to Check
Pod status: **OOMKilled**	Exceeded memory limit.	**kubectl describe pod** (check Last State, Reason).
High CPU, slow app	CPU throttling (hit the limit).	**kubectl top pod** (check CPU usage vs limit).
Pod Pending	Not enough node capacity for requests.	**kubectl describe pod** (look for FailedScheduling event).

Junior Engineer Tip

Set realistic requests (for scheduling) and limits (to prevent resource abuse). Monitor usage, and investigate OOMKilled or throttling events early.

Health Checks: Readiness, Liveness, and Startup Probes

Overview

- Kubernetes uses probes (health checks) to monitor application health and availability.
- They tell Kubernetes when an app is ready for traffic, when to restart it, and when to give it more time to start.

The Three Probes

Readiness Probe	Checks if the app is ready to receive traffic. If it fails, the Pod remains running but is removed from Service traffic.
Liveness Probe	Checks if the app is still healthy. If it fails, Kubernetes restarts the container.
Startup Probe	Gives the app more time to start (useful for slow-starting applications) before liveness and readiness checks begin.

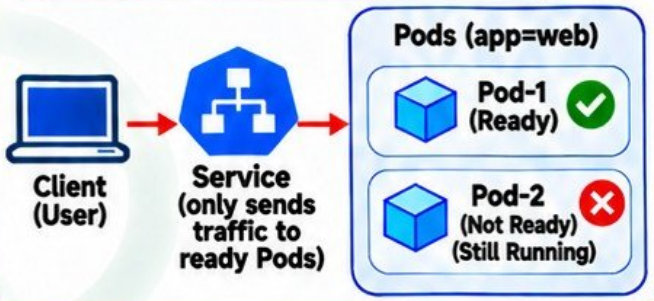
Why It Matters

- Keeps traffic only on healthy, ready Pods.
- Automatically restarts unhealthy applications.
- Prevents false restarts for applications that take longer to start.
- Improves availability and user experience.

How It Works



Request Flow with Readiness



Probe Configuration (Example)

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 2
  template:
    spec:
      containers:
      - name: web
        image: nginx:1.27
        ports:
        - containerPort: 80
        startupProbe:
          httpGet:
            path: /health
            port: 80
          failureThreshold: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: 80
          initialDelaySeconds: 5
          periodSeconds: 10
        livenessProbe:
          httpGet:
            path: /live
            port: 80
          initialDelaySeconds: 15
          periodSeconds: 10
```

Failure Scenarios and Impact

Incorrect Liveness Probe (Restart Loop)

- If the liveness probe is too strict (e.g., wrong path or short timeout), Kubernetes keeps restarting the container.
- Result: A restart loop (CrashLoopBackOff), and the app never becomes stable.

Readiness Failure (Traffic Blocked)

- If the readiness probe fails, the Pod stays running but is removed from the Service endpoints.
- Result: The Pod is healthy but will not receive any user traffic until it passes the check.

Hands-On: Inspect and Test

`$ kubectl get pods`	Check Pod status and restarts.
`$ kubectl describe pod web-app-xxx`	View probe configuration and recent events.
`$ kubectl get endpoints web-app`	See which Pods are receiving Service traffic.
`$ kubectl logs web-app-xxx`	Check application logs for probe-related issues.

Key Takeaways

- Use a startup probe for slow-starting apps to avoid premature restarts.
- Use a readiness probe to control when the Pod receives traffic.
- Use a liveness probe to restart truly unhealthy applications, not for temporary issues.
- Tune delay and threshold values based on your application's real behavior.

Rolling Updates and Rollbacks

Overview

- Update application images in a controlled and safe way.
- Keeps the application available during the update.
- Quickly roll back to a stable version if the new release has issues.

Key Terms

Rolling Update	Gradually replaces old Pods with new ones (without downtime).
Rollout	The update process for a Deployment.
History	Records of past deployments (revisions).
Rollback	Revert to a previous, known good version.

Why It Matters

- Avoids service downtime and user disruption.
- Lets you test a new version in production safely.
- Provides a fast recovery (rollback) if the new release fails.
- Supports safer, repeatable releases in production.

Rolling Update: How It Works



Rolling-Update Strategy

Type	RollingUpdate (default for Deployments).
maxSurge	How many extra Pods can be created above the desired replica count (e.g., 25%).
maxUnavailable	How many Pods can be temporarily unavailable during the update (e.g., 25%).

Hands-On: Update, Check, and Rollback

`$ kubectl set image deployment/web \ web=nginx:1.27`	Update the Deployment image to a new version.
`$ kubectl rollout status \ deployment/web`	Watch the rolling update progress.
`$ kubectl rollout history deployment/web`	Show deployment history (revisions).
`$ kubectl rollout undo deployment/web`	Rollback to the previous working version.
`$ kubectl rollout undo deployment/web --to-revision=1`	Rollback to a specific revision (e.g., revision 1).

Deployment Example (Rolling Update)

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
          ports:
          - containerPort: 80
```

Detecting a Failed Release



- The rollout gets stuck (e.g., Pods not ready).
- New Pods keep restarting (CrashLoopBackOff).
- Readiness or liveness probes are failing.
- Application errors or increased failure rate (e.g., HTTP 5xx).

Post-Deployment Verification

- Confirm rollout success: `kubectl rollout status deployment/web`.
- Check Pod health: `kubectl get pods -l app=web`.
- Test the application: `curl http://<service-ip>` or via Ingress.
- Review logs: `kubectl logs deployment/web`.
- Verify metrics (CPU, memory, errors) in your monitoring tool.

Key Takeaways

- Use rolling updates to release new versions without downtime.
- Monitor the rollout, verify the application, and be ready to roll back if needed.
- A failed release can be quickly reverted with `kubectl rollout undo`.

Namespaces, Labels, and Resource Organization

Overview

- Organize and group Kubernetes resources for teams, environments, or applications.
- Use namespaces to separate resources logically.
- Use labels, selectors, and annotations to identify, filter, and manage resources.

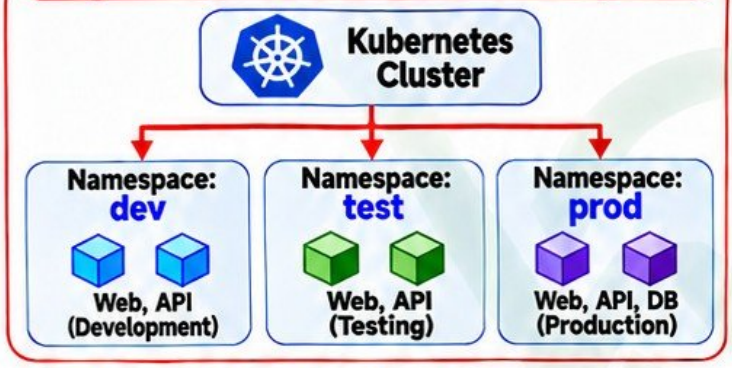
Key Terms

Namespace	A logical group for Kubernetes resources (e.g., dev, test, prod).
Label	A key/value pair used to identify and group resources (e.g., app=web).
Selector	A filter that finds resources based on labels (e.g., app=web).
Annotation	Extra metadata for tools, tracking, or documentation (e.g., owner=platform-team).

Why It Matters

- Keeps resources organized (e.g., dev, test, prod).
- Helps teams work independently without name conflicts.
- Makes it easy to select, filter, and manage groups of resources.
- Supports automation, policies, and better visibility.

Organizing Resources with Namespaces



Labels, Selectors, and Annotations

Labels
- Identify and group resources.
- Example: app=web, tier=frontend, env=prod.

Selectors
- Find resources based on labels.
- Used by Services, Deployments, etc.
- Example: selector: app=web.

Annotations
- Store non-identifying metadata (e.g., owner, purpose, link).
- Commonly used by tools and for documentation.

List and Filter Workloads

`$ kubectl get ns`	List all namespaces.
`$ kubectl get pods -n dev`	List Pods in the dev namespace.
`$ kubectl get deploy -n prod`	List Deployments in prod.
`$ kubectl get pods -l app=web`	List Pods with label app=web (all namespaces).
`$ kubectl get pods -n prod -l tier=frontend`	Filter Pods in prod with tier=frontend.

Namespace Operations (Hands-On)

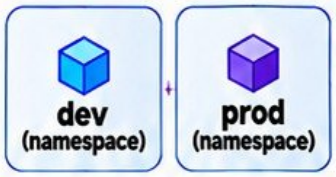
`$ kubectl create ns dev`	Create a namespace.
`$ kubectl config set-context \ --current --namespace=dev`	Set the current namespace for your session.
`$ kubectl get all -n dev`	List all resources in dev (Pods, Services, Deployments, etc.).
`$ kubectl delete ns test`	Delete a namespace (and all its resources).

Example: Deployment with Labels and Annotations

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: prod
  labels:
    app: web
    tier: frontend
  annotations:
    owner: "platform-team"
    purpose: "frontend service"
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        tipr: web
        tier: frontend
    spec:
      containers:
      - name: web
        image: nginx:1.27
```

YAML

Important: Namespace Boundaries



Organizational boundary (separation, grouping, quota)

Namespaces are NOT a complete security isolation.

- They separate and organize resources (logical boundary).
- By default, users with cluster access can still see resources across namespaces.
- Use RBAC (Roles and RoleBindings), NetworkPolicies, and other controls for real security isolation.

Key Takeaways



- Use namespaces to organize teams, environments, and apps.
- Use labels and selectors to find and manage related resources.
- Use annotations for extra metadata (e.g., owner, purpose, links).
- Remember: namespaces provide organization, not full security.

Kubernetes Security: ServiceAccounts and RBAC

Overview

- Control what users and workloads can do in the Kubernetes API.
- Use ServiceAccounts (SA) for applications, and RBAC (Roles and RoleBindings) for authorization.
- Follow least privilege: only grant the minimum permissions needed.

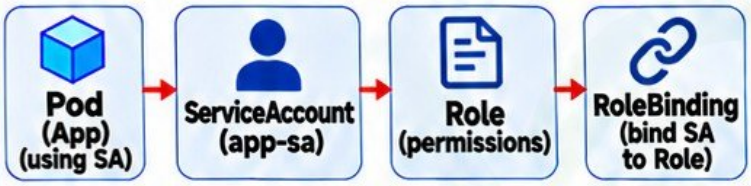
Key Terms

Identity	Who you are (user, system, or ServiceAccount).
Authentication	Verifies the identity (e.g., token, certificate).
Authorization	What actions you are allowed to do (e.g., get, list, create).
ServiceAccount	A non-human identity for workloads (Pods) to access the Kubernetes API.
RBAC	Role-based access control using Roles, ClusterRoles, and Bindings.

Why It Matters

- Limits the impact of a compromise.
- Prevents applications from having unnecessary cluster-level access.
- Enforces separation of duties and safer operations.
- Reduces the risk of data leakage and cluster takeover.

How ServiceAccounts and RBAC Work



Pod uses the ServiceAccount token to call the Kubernetes API with the permissions defined in the Role.

RBAC Building Blocks

Role	Defines permissions within a namespace (e.g., get, list Pods).
ClusterRole	Defines permissions cluster-wide (e.g., list nodes, access all namespaces).
RoleBinding	Binds a Role to a ServiceAccount, user, or group (namespace scope).
ClusterRoleBinding	Binds a ClusterRole (cluster-wide).

Example: ServiceAccount + Role + RoleBinding

YAML

```yaml
# 1. ServiceAccount
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: dev
---
# 2. Role (allow read access to Pods)
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
# 3. RoleBinding (bind the Role to the SA)
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: dev
subjects:
- kind: ServiceAccount
  name: app-sa
  namespace: dev
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Check Permissions with kubectl auth can-i

`$ kubectl auth can-i get pods -n dev --as=system:serviceaccount:dev:app-sa`	Check if the SA can read Pods.
`$ kubectl auth can-i create pods -n dev --as=system:serviceaccount:dev:app-sa`	Check if the SA can create Pods.
`$ kubectl auth can-i list nodes --as=system:serviceaccount:dev:app-sa`	Test cluster-level access (should be no).

Security Risks to Avoid

Unnecessary API Access
- Giving broad permissions (e.g., cluster-admin) to application Pods.
- Increases the blast radius if the Pod is compromised.

Privileged Containers
- Running with privileged: true (full host access).
- Can be used to escape the container and take over the node.

Exposed Credentials
- Hardcoding tokens, keys, or cluster credentials in images or ConfigMaps.
- Can be stolen and used to access or control the cluster.

Key Takeaways



- Use a dedicated ServiceAccount for each application (not the default).
- Grant only the minimum permissions required (least privilege).
- Regularly verify access with kubectl auth can-i.
- Avoid privileged containers and never expose credentials in images or source code.

Scheduling and Scaling: Where Pods Run

Overview

- Control where Pods run in the cluster and how they scale to handle load.
- Use node selection, taints, tolerations, affinity, and anti-affinity to influence placement.
- Use Horizontal Pod Autoscaling (HPA) to adjust the number of Pods automatically.

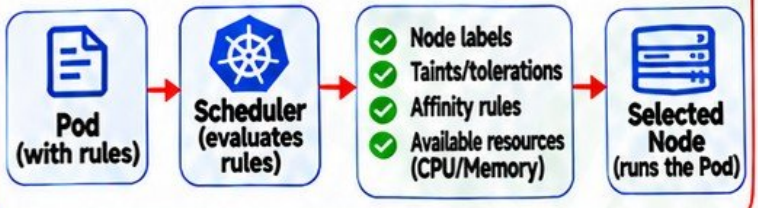
Key Terms

Node Selection	Place Pods on specific nodes using labels (nodeSelector).
Taint	Mark a node to repel Pods unless they have a toleration.
Toleration	A rule that allows a Pod to run on a tainted node.
Affinity	Prefer or require Pods to run on certain nodes or with related Pods.
Anti-Affinity	Keep Pods apart (e.g., one Pod per node for high availability).
HPA	Automatically scales Pods based on metrics (e.g., CPU, memory).

Why It Matters

- Improves performance and reliability.
- Keeps critical workloads on the right nodes (e.g., GPU or high-memory).
- Helps spread Pods for resilience (anti-affinity).
- Automatically scales apps with demand (HPA).

How Kubernetes Chooses a Node



Node Placement Controls

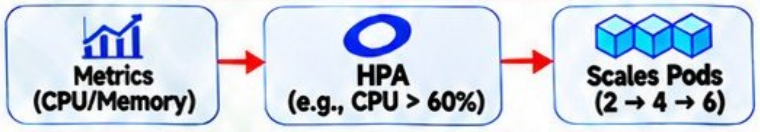
nodeSelector	Run Pod on nodes with specific labels (e.g., disktype=ssd).
Taints	Mark nodes (e.g., dedicated=gpu:NoSchedule).
Tolerations	Allow Pod to run on a tainted node (e.g., dedicated=gpu:NoSchedule).
Affinity	Attract Pods to certain nodes (e.g., same zone).
Anti-Affinity	Spread Pods across different nodes (for high availability).

Example: Affinity and Toleration

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 2
  template:
    metadata:
      labels:
        app: web
    spec:
      tolerations:
      - key: "dedicated"
        operator: "Equal"
        value: "gpu"
        effect: "NoSchedule"
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
            - matchExpressions:
              - key: zone
                operator: In
                values: ["us-east-1a"]
```

Horizontal Pod Autoscaling (HPA)



HPA Example

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 6
  metric:
    - type: Resource
      resource:
        name: cpu
        trget:
        type:   Utilization
        averageUtilization: 60
```

HPA Requirements

- Metrics source (e.g., Metrics Server) must be installed.
- Target Deployment/ReplicaSet to scale.
- A metric to watch (e.g., CPU utilization).
- Define minimum and maximum replicas.

When Scheduling Fails



- Pod stays in **Pending** state.
- Common reasons: insufficient CPU/Memory, no matching node labels, or taints without tolerations.
- Check with: **kubectl get pods** and **kubectl describe pod <pod-name>**.

Important Clarifications

More Replicas ≠ More Capacity	Adding replicas creates more Pods, but it does not add CPU, memory, or nodes to your cluster.
To Increase Capacity	Add more nodes (e.g., scale your cluster or node group) or use a bigger node size.

Key Takeaways

- Use labels, taints, tolerations, and (anti-)affinity to control Pod placement.
- Use HPA to scale based on real metrics (e.g., CPU, memory).
- A Pending Pod means the scheduler cannot find a suitable node.
- Scaling replicas increases the number of Pods, not the cluster's total capacity.

Troubleshooting Pods: CrashLoopBackOff and ImagePullBackOff

Overview

- Learn a practical workflow to diagnose and fix common Pod failures.
- Focus on two frequent states: CrashLoopBackOff and ImagePullBackOff.
- Use kubectl get, describe, logs, and events to find the root cause and fix it safely.

Key Terms

CrashLoopBackOff	The container keeps starting and crashing repeatedly.
ImagePullBackOff	Kubernetes cannot download the container image.
Events	A timeline of what happened to the Pod (e.g., pull, start, fail).
Logs	The container's output (helps find the crash cause).

Diagnostic Workflow

1. Check Pod status: `kubectl get pods`
2. Describe the Pod: `kubectl describe pod <pod>`
3. View container logs: `kubectl logs <pod> [-c <container>]`
4. Check events: `kubectl get events --sort-by=.lastTimestamp`
5. Fix the root cause (config, image, code, etc.).
6. Verify the fix: `kubectl get pods -w`

CrashLoopBackOff: Container Keeps Crashing

CrashLoop BackOff

What It Means	The container starts, fails, and Kubernetes keeps restarting it (with backoff).
Common Causes	■ Application error (runtime crash). ■ Wrong command or arguments. ■ Missing config, secret, or file. ■ Out of memory (OOMKilled).
Safe Fixes	✓ Check logs for the actual error. ✓ Fix the application/configuration. ✓ Increase memory limit if needed. ✓ Redeploy and verify.

ImagePullBackOff: Image Cannot Be Downloaded

ImagePull BackOff

What It Means	Kubernetes tried to pull the image but failed, and will retry periodically.
Common Causes	■ Wrong image name or tag. ■ Image does not exist in registry. ■ Private registry without credentials. ■ Network or registry access issues.
Safe Fixes	✓ Verify the image name and tag. ✓ Add or fix imagePullSecrets. ✓ Check registry access (permissions, network). ✓ Update the Deployment and verify.

Example: Findings and Evidence

CrashLoopBackOff Evidence

```
$ kubectl get pods
NAME        READY   STATUS             RESTARTS
web-7c8d5   0/1     CrashLoopBackOff   5

$ kubectl logs web-7c8d5
Error: Cannot find module
'/app/server.js'
```

Reason: The application crashes because the entry file is missing.

ImagePullBackOff Evidence

```
$ kubectl get pods
NAME          READY   STATUS             RESTARTS
api-6f9d7     0/1     ImagePullBackOff   0

$ kubectl describe pod api-6f9d7
Events:
  Warning    Failed      ...
    ErrImagePull: pull access denied
    for myrepo/api:1.0
  Warning    Failed      ...
    ImagePullBackOff: Back-off
    pulling image "myrepo/api:1.0"
```

Reason: Kubernetes cannot pull 'myrepo/api:1.0' (private or not found).

Useful Commands (Quick Reference)

`$ kubectl get pods`	Check Pod status.
`$ kubectl describe pod <pod>`	See detailed info and events.
`$ kubectl logs <pod>`	View container logs (add -c <container> if needed).
`$ kubectl get events --sort-by=.lastTimestamp`	See recent cluster events (reasons for failures).
`$ kubectl rollout restart deployment/<name>`	Restart the workload after a fix.
`$ kubectl get pods -w`	Watch Pod status until it becomes Running.

Why This Happens Often

- Misconfiguration (wrong command, path, or env).
- Image issues (wrong tag, missing image, or access denied).
- Resource problems (e.g., OOM kills).
- Dependencies not available (DB, secrets, config).

Verification After the Fix

- ✓ Pod status is Running and READY (e.g., 1/1).
- ✓ No new errors in container logs.
- ✓ No repeated failure events.
- ✓ Application endpoint responds (e.g., curl or Ingress).

Key Takeaways

- CrashLoopBackOff = container crashes repeatedly (check logs and fix the app/config).
- ImagePullBackOff = image cannot be downloaded (check image name, registry, and credentials).
- Always use kubectl get, describe, logs, and events to find the real cause.
- Fix the root cause, redeploy, and verify the Pod is Running and Ready.

Troubleshooting Services: Application Is Running but Unreachable

Overview

- Investigate why an application is running but cannot be reached.
- Check Service selectors, EndpointSlices, target ports, readiness, DNS, and network policies.
- Follow a step-by-step workflow to find the first failing stage.

Key Terms

Service	A stable network entry point for a set of Pods.
Selector	Label filter that tells the Service which Pods to use.
EndpointSlice	The list of Pod IPs and ports backing a Service.
Target Port	The port on the Pod that the Service sends traffic to.
Readiness	A probe that determines if a Pod can receive traffic.
NetworkPolicy	Rules that allow or block network traffic to/from Pods.

Why It Matters

- Services depend on correct labels, ports, and readiness.
- The Service can exist but route to zero Pods (no endpoints).
- DNS must resolve the Service name to reach it inside the cluster.
- Network policies can block traffic even when Pods and Services look correct.

Incident Scenario (Realistic)



Problem: The web application Pods are running, but users cannot reach it via the Service URL (http://web-svc:80).

Environment

- Namespace: prod
- Deployment: web
- Service: web-svc
- Expected: Users can access the application.
- Actual: Connection times out or returns "503 Service Unavailable".

Diagnostic Workflow (Find the First Failure)

1. Check Pods (health)
2. Check Service (selectors, ports)
3. Check EndpointSlices (are Pods listed?)
4. Check Readiness (are Pods ready?)
5. Check DNS & Network Policies (can traffic flow?)

Step-by-Step Investigation (Commands and Evidence)

1 `$ kubectl get pods -n prod`

```
NAME                   READY   STATUS    RESTARTS
web-6d7c9d9f7-2x9kd    1/1     Running   0
web-6d7c9d9f7-vt8nm    1/1     Running   0
```

2 `$ kubectl get svc web-svc \ -n prod`

```
NAME      TYPE        CLUSTER-IP      PORT(S)
web-svc   ClusterIP   10.96.44.12     80/TCP
```

3 `$ kubectl describe svc web-svc \ -n prod`

```
Selector:     app=webapp
Port:         80/TCP
TargetPort:   8080/TCP
```

4 `$ kubectl get endpointslices \ -n prod -l kubernetes.io/service-name=web-svc`

```
NAME              ENDPOINTS
web-svc-pk914     <none>
```

5 `$ kubectl get pods -n prod --show-labels`

```
NAME                   LABELS
web-6d7c9d9f7-2x9kd    app=web, tier=frontend
web-6d7c9d9f7-vt8nm    app=web, tier=frontend
```

Analysis and Root Cause

Observation

- Pods are Running (healthy).
- Service selector is app=webapp.
- Pods are labeled app=web (mismatch).
- EndpointSlice has <none> (no backends).

First Failing Stage

Service → Endpoint discovery (selector does not match Pod labels).

Root Cause

The Service selector (app=webapp) does not match the Pods' label (app=web), so the Service has no endpoints and cannot route traffic.

Fix and Verification

Fix (Choose One)

- Update the Service selector to match the Pods:

```
$ kubectl patch svc web-svc -n prod \
-p "{"spec":{"selector":{"app":"web"}}}"
```

- Or update the Pods' label to match the Service (and restart):

```
$ kubectl label pod -n prod -l app=web \
app=webapp --overwrite
```

Verify

- `$ kubectl get endpointslices -n prod \ -l kubernetes.io/service-name=web-svc` → Endpoints are now listed.
- `$ curl -I http://web-svc:80` → HTTP 200 OK (or expected response).
- `$ kubectl get svc web-svc -n prod` → Confirm selector and ports are correct.

Other Possible Causes to Check

Target Port	Service targetPort does not match the container port (e.g., Service 80 → Pod 8080).
Readiness	Pods are not Ready, so they are not added to endpoints.
DNS	Service name not resolving (e.g., typo or wrong namespace).
Network Policy	A NetworkPolicy is blocking traffic to the Pods.

Key Takeaways

- When an app is running but unreachable, start with the Service → Endpoints → Readiness flow.
- Always check that Service selectors match the Pod labels.
- Verify target ports, DNS resolution, and network policies.
- Identify the first failing stage, fix it, and then verify end-to-end access.

Kubernetes Observability: Logs, Metrics, and Events

Overview

- Understand the key observability signals in Kubernetes: logs, metrics, traces, and events.
- Learn practical kubectl commands to view and analyze them.
- Use these signals to check application health, respond to incidents, and monitor production workloads.

Key Terms

Logs	Text records from the application or container (e.g., errors, requests).
Metrics	Time-series numeric data (e.g., CPU, memory, request rate).
Traces	End-to-end view of a request across multiple services (e.g., latency, where it slowed).
Events	Kubernetes-recorded actions in the cluster (e.g., Pod started, failed, rescheduled).

Why It Matters

- Gives a complete view of application and cluster health.
- Helps you quickly find the root cause during an incident.
- Supports capacity planning and performance tuning.
- Provides evidence and timelines for post-incident reviews.

The Four Signals at a Glance

Logs	What happened (e.g., errors, info, requests).
Metrics	How the system is performing (e.g., CPU, memory, traffic).
Traces	How a request flows across services (e.g., latency, bottlenecks).
Events	What Kubernetes did (e.g., scheduled, restarted, failed).

Practical Commands (with Example Output)

```
$ kubectl logs pod/web-7c8d5
2026-11-04 07:21:10  INFO   Server started
2026-11-04 07:21:15  ERROR  DB connection failed
```

```
$ kubectl top pods
NAME        CPU(cores)   MEMORY
web-7c8d5   120m         256Mi
web-7c8d6   110m         248Mi
```

```
$ kubectl top nodes
NAME       CPU(cores)   MEMORY
worker-1   35%          72%
worker-2   28%          61%
```

```
$ kubectl get events --sort-by=.lastTimestamp
LAST SEEN   TYPE      REASON      OBJECT
2m          Warning   BackOff     pod/web-7c8d5
5m          Normal    Scheduled   pod/web-7c8d5
```

Observability in Action: From Signal to Insight



Prometheus and Grafana (Quick Intro)

Prometheus

- Collects and stores time-series metrics from Kubernetes and applications.
- Commonly used for alerting (e.g., high CPU, pod restarts).

Grafana

- Visualizes metrics from Prometheus (dashboards, trends, alerts).
- Helps you monitor application and cluster health at a glance.

Real-World Use Case: Investigating a Slow Application

1. **Check Pod health:** `$ kubectl get pods` → Pods are Running.
2. **Review logs for errors:** `$ kubectl logs pod/web-7c8d5` → Found slow DB queries.
3. **Check resource usage:** `$ kubectl top pods` → High CPU (450m) and memory (380Mi).
4. **Check recent events:** `$ kubectl get events --sort-by=.lastTimestamp` → No crashes, but high restarts earlier.
5. **Take action:**
   - Optimize the query, increase resources, or scale the app.
   - Verify again with logs, metrics, and user response.

Key Takeaways



- Logs = details, Metrics = performance, Traces = request journey, Events = cluster actions.
- Use kubectl logs, kubectl top, and kubectl get events for quick diagnosis.
- Use Prometheus + Grafana for ongoing monitoring and alerting.
- Combine these signals to confirm the problem, not just guess the cause.

Final Practical Challenge: Investigate a Broken Deployment

Incident Report (Scenario)



- **Time:** Nov 4, 2026, 10:15 UTC
- **Application:** webapp
- **Environment:** Production (prod)
- **Issue:** Users report "Service Unavailable (503)" on https://app.veriqta.com.
- **Change:** New image (v2.1.0) was deployed 10 minutes earlier.

Your Challenge



Use the evidence below to:

- Identify symptoms and affected components.
- Determine which failure occurred first.
- Select appropriate diagnostic commands.
- Explain the root cause using evidence.
- Propose the smallest safe corrective action.
- Verify application recovery.
- Identify one preventive improvement.

Evidence (Provided During the Incident)

1 Pod Status

```
$ kubectl get pods -n prod
```

NAME	READY	STATUS	RESTARTS	AGE
webapp-7c6d5f9c8d-6j2m7	0/1	CrashLoopBackOff	7	9m
webapp-7c6d5f9c8d-k8p4z	0/1	CrashLoopBackOff	7	9m

2 Deployment Details

```
$ kubectl get deploy webapp -n prod
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
webapp	0/2	2	0	9m

3 Service Configuration

```
$ kubectl get svc webapp -n prod -o wide
```

NAME	TYPE	CLUSTER-IP	PORT(S)	SELECTOR
webapp	ClusterIP	10.96.55.20	80/TCP	app=webapp

4 Recent Events

```
$ kubectl get events -n prod --sort-by=.lastTimestamp
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
9m	Normal	Pulling	pod/webapp-7c6d5	Pulling image "veriqta/webapp:v2.1.0"
8m	Normal	Pulled	pod/webapp-7c6d5	Successfully pulled image
8m	Normal	Created	pod/webapp-7c6d5	Created container webapp
8m	Normal	Started	pod/webapp-7c6d5	Started container webapp
7m	Warning	BackOff	pod/webapp-7c6d5	Back-off restarting failed container

5 Application Logs

```
$ kubectl logs pod/webapp-7c6d5f9c8d-6j2m7 -n prod --previous
```

```
2026-11-04 10:08:21  INFO   Starting webapp v2.1.0 ...
2026-11-04 10:08:22  ERROR  Failed to connect to database
2026-11-04 10:08:22  ERROR  dial tcp 10.96.66.10:5432: connect:
                            connection refused
2026-11-04 10:08:22  FATAL  Exiting (code=1)
```

Investigation Tasks (What to Do)

1. List the symptoms and affected components.
2. Determine which failure occurred first (use events/logs).
3. Choose and run the right diagnostic commands (e.g., get pods, logs, events, svc, endpoints).
4. Explain the root cause with evidence.
5. Propose the smallest safe fix (minimum change).
6. Verify the application is working again.
7. Suggest one preventive improvement.

Key Commands (Quick Reference)

```
$ kubectl get pods -n prod
$ kubectl describe pod <pod> -n prod
$ kubectl logs <pod> -n prod
$ kubectl get deploy webapp -n prod
$ kubectl get svc webapp -n prod
$ kubectl get endpointslices -n prod
$ kubectl get events -n prod --sort-by=.lastTimestamp
$ kubectl rollout restart deploy/webapp -n prod
```

Investigation Guide / Answer Key (Evidence-Based)

Symptoms & Affected
- Webapp is unavailable (503).
- Affected: Pods (CrashLoopBackOff) and Deployment (0/2 ready).

First Failure
- Database connection failed (from logs) → container exited (code=1) → then CrashLoopBackOff (from events).

Root Cause
- The new webapp (v2.1.0) cannot connect to the database (10.96.66.10:5432) — connection refused (DB not running, wrong address, or network policy blocking).

Corrective Action
- Restore database connectivity, e.g., update the DB service address/secret or ensure the DB is running, then redeploy or restart the app:
  `$ kubectl rollout restart deploy/webapp -n prod`

Verify Recovery
- `$ kubectl get pods -n prod` → Pods READY 2/2 and STATUS Running.
- `$ curl -I http://app.veriqta.com` → HTTP 200 OK.

Preventive Improvement
- Add a readiness probe and pre-deployment check (e.g., test DB connectivity), and use a rollout with monitoring/alerting.

Key Takeaways



- Follow evidence, not guesswork.
- Logs and events reveal the first failure.
- Service, Deployment, and Pod health are connected.
- Fix the root cause with the smallest safe change.
- Verify, then add a preventive control (e.g., readiness probe, monitoring, or pre-deploy checks).